

PORTADA

Arquitectura de Software

Semana 9 — Estilos Arquitectónicos

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

**Ya viste 15 patrones.
¿Cuántos estilos hay detrás?**



TRANSICIÓN

Antes de seguir, hace falta un mapa

En las últimas seis sesiones vieron catorce patrones puntuales del catálogo POSA (Buschmann et al.): Pipes & Filters, Blackboard, Broker, MVC, Batch, Cliente/Servidor N-capas, Layers, Microkernel, SOA, Hexagonal, Microservicios, WebHooks, BFF y Circuit Breaker — más SOFEA como variante adicional, quince en total. Cada uno resolvía un problema concreto, en su propia sesión, sin conectarlo todavía con los demás.

Hoy no vemos un patrón nuevo — vemos el mapa completo: los estilos arquitectónicos, las categorías mayores dentro de las cuales viven todos esos patrones.

CASO · FICHA TÉCNICA

Shaw y Garlan, 1994–1996

Autoras	Mary Shaw y David Garlan
Universidad	Carnegie Mellon University (CMU)
Trabajo fundacional	"An Introduction to Software Architecture", reporte técnico de CMU, 1994
Consolidación	"Software Architecture: Perspectives on an Emerging Discipline", Prentice Hall, 1996
Objetivo	Proponer una taxonomía de estilos arquitectónicos para clasificar sistemas de software según patrones estructurales recurrentes a nivel de sistema completo
Conexión	Los mismos autores de los "Conectores y Protocolos" vistos en la Semana 5

Sí, es el mismo Garlan de la Semana 5 — no repetimos ese contenido, solo lo referenciamos como parte del mismo trabajo conjunto con Shaw.

CASO · POR QUÉ CREAR UNA TAXONOMÍA

Ordenar el vocabulario de la disciplina

A comienzos de los años 90 la arquitectura de software crecía sin vocabulario común: cada equipo describía su sistema con sus propios términos, y era difícil comparar dos arquitecturas o reconocer que dos sistemas, aunque contaran historias distintas, compartían la misma forma estructural.

Shaw y Garlan propusieron clasificar los sistemas según su estilo arquitectónico: una categoría que restringe qué tipos de elementos de diseño existen (componentes, conectores) y qué formas de interacción entre ellos son válidas, a nivel de todo el sistema.

- Permite comparar sistemas distintos usando el mismo lenguaje.
- Facilita razonar sobre los atributos de calidad típicos de cada estilo.
- Sirve de punto de partida para diseñar sistemas nuevos, en vez de empezar desde cero.

CASO · LAS CATEGORÍAS PROPUESTAS

Cinco grandes familias de estilos

Categoría	Ejemplos según Shaw y Garlan
Sistemas de flujo de datos	Pipe-filter, procesamiento por lotes (batch)
Sistemas de datos compartidos	Repositorio, pizarra (blackboard)
Sistemas de llamada y retorno	Capas (layers), orientado a objetos, cliente/servidor
Sistemas independientes	Procesos comunicantes, orientado a eventos
Sistemas orientados a datos virtuales	Intérprete

Esta es la clasificación original de 1994–1996; el resto de la sesión la usa para ubicar todo lo que ya vieron.

TRANSICIÓN

De la taxonomía general a la distinción precisa

Con el mapa de Shaw y Garlan a la vista surge la pregunta que da nombre a esta sesión: si ya conocen quince patrones concretos del catálogo POSA, ¿en qué se diferencian exactamente de un "estilo"?

La respuesta es de nivel de abstracción, no de contenido — y es clave para no confundir ambos términos de aquí en adelante.

CONCEPTO

Estilo arquitectónico vs. patrón

	Estilo arquitectónico	Patrón (catálogo POSA)
Nivel de abstracción	Categoría amplia, a nivel de sistema completo	Solución concreta a un problema recurrente
Qué restringe	El vocabulario general de elementos de diseño y sus formas válidas de interacción	Una estructura específica de componentes y conectores
Ejemplo	"Sistemas de llamada y retorno"	Layers, Broker, MVC
Relación entre ambos	Un estilo agrupa muchos patrones posibles; un patrón es una instancia concreta de un estilo	

En corto: el estilo dice bajo qué reglas generales se relacionan los componentes del sistema; el patrón dice exactamente cómo resolverlo en un caso concreto.

CONCEPTO · EJEMPLO

Un patrón, ubicado dentro de su estilo

Tomen Pipes & Filters, visto en la Semana 6: cada filtro procesa datos y los pasa al siguiente a través de una tubería, sin conocer el resto del pipeline.

Ese patrón no inventa una forma nueva de relacionarse — es una **instancia concreta** del estilo de flujo de datos que ya proponía la taxonomía de Shaw y Garlan: componentes que transforman datos y conectores que los transportan entre ellos, sin más contacto que ese flujo.

Lo mismo aplica a los otros catorce patrones que ya vieron: cada uno vive dentro de una de las cinco familias de estilos — y organizar ese mapa completo es el objetivo del resto de la sesión.

CONCEPTO · CATEGORÍAS DE SHAW Y GARLAN

Flujo de datos y datos compartidos

Estilo	Idea central	Patrones POSA que ya vieron
Flujo de datos	Los componentes transforman datos y los pasan al siguiente mediante conectores de transporte; no hay estado compartido entre ellos	Pipes & Filters, procesamiento Batch
Datos compartidos	Los componentes interactúan indirectamente, leyendo y escribiendo en un almacén central compartido	Blackboard (pizarra)

En flujo de datos los componentes se conectan entre sí; en datos compartidos se conectan a través de un almacén común, nunca directamente.

CONCEPTO · CATEGORÍAS DE SHAW Y GARLAN

Llamada y retorno — la familia más grande

Es, con diferencia, la categoría con más patrones del catálogo POSA que ya vieron: un componente invoca explícitamente a otro y espera su respuesta, en una relación de control jerárquica y sincrónica.

- Incluye desde un simple objeto que llama a un método de otro, hasta sistemas distribuidos donde un cliente invoca a un servidor a través de la red.
- Layers, MVC, Cliente/Servidor N-capas, Broker, Microkernel, Hexagonal, SOA, Microservicios y BFF — nueve de los quince patrones que ya vieron — pertenecen a esta familia.

El detalle patrón por patrón viene en la tabla de mapeo completo, dos diapositivas más adelante.

CONCEPTO · CATEGORÍAS DE SHAW Y GARLAN

Sistemas independientes y datos virtuales

Estilo	Idea central	Patrones POSA que ya vieron
Sistemas independientes	Componentes autónomos que no comparten control directo; se coordinan mediante eventos o mensajes, sin llamada síncrona	WebHooks, Circuit Breaker (como mecanismo de control entre componentes independientes)
Orientado a datos virtuales	Un componente (el intérprete) interpreta y ejecuta una representación de otro programa, en vez de ejecutarlo directamente	No visto en el catálogo POSA de este curso — ejemplo típico: una máquina virtual o un motor de reglas

Orientado a eventos: los componentes se notifican mediante mensajes o eventos, sin que quien emite el evento conozca ni controle a quien lo recibe.

CONCEPTO · EL MAPA COMPLETO

Los 15 patrones POSA, por estilo (1/2)

Patrón (catálogo POSA)	Estilo arquitectónico
Pipes & Filters	Flujo de datos
Batch (procesamiento por lotes)	Flujo de datos
Blackboard	Datos compartidos
Broker	Llamada y retorno
MVC	Llamada y retorno
Cliente/Servidor N-capas	Llamada y retorno
Layers	Llamada y retorno
Microkernel	Llamada y retorno

Patrones vistos en las semanas 6 y 7.

CONCEPTO · EL MAPA COMPLETO

Los 15 patrones POSA, por estilo (2/2)

Patrón (catálogo POSA)	Estilo arquitectónico
SOA	Llamada y retorno
Hexagonal	Llamada y retorno
Microservicios	Llamada y retorno
WebHooks	Independiente / orientado a eventos
BFF	Llamada y retorno (variante orientada al cliente)
Circuit Breaker	Independiente (mecanismo de control entre componentes)
SOFEA	Llamada y retorno (variante con orquestación en el cliente)

Patrones vistos en las semanas 8 y 9. SOFEA no inventa una relación nueva entre componentes: sigue siendo llamada y retorno, pero mueve la orquestación de esas llamadas al cliente en vez de dejarla en el servidor.

TENSIÓN CONCEPTUAL

Ningún sistema real usa un solo estilo puro

La taxonomía separa estilos con fronteras nítidas, pero los sistemas reales casi nunca calzan en una sola categoría — combinan varios estilos en distintas partes.

Ejemplo: un sistema de microservicios (llamada y retorno entre servicio y servicio vía REST o gRPC) casi siempre incorpora también mensajería asíncrona orientada a eventos (independiente) para desacoplar procesos que no necesitan respuesta inmediata, y a menudo persiste datos en un almacén compartido (datos compartidos) para reportes o auditoría.

Reconocer el estilo dominante de un sistema no significa que sea el único presente — significa identificar qué estilo gobierna las decisiones estructurales más importantes.



CONCEPTO

Elegir un estilo es una decisión de negocio, no solo técnica

Como se vio en "Diseño vs Arquitectura" (Semana 3) con el caso del Aeropuerto de Denver, las decisiones arquitectónicas son las más costosas de revertir — y elegir un estilo arquitectónico es la decisión más temprana y más cara de todas: define cómo se van a relacionar los componentes del sistema completo, mucho antes de escribir la primera línea de un módulo concreto.

Ese estilo no debería elegirse por preferencia personal del equipo ni por la tecnología de moda, sino por los **atributos de calidad prioritarios del sistema** — los mismos que se formalizaron con ISO 25010 en la Semana 4: si el sistema prioriza escalabilidad y despliegue independiente, un estilo de llamada y retorno distribuido (microservicios) tiene sentido; si prioriza consistencia fuerte y simplicidad operativa, un estilo de datos compartidos centralizado puede ser mejor opción.

El estilo correcto no es el más nuevo ni el más popular: es el que mejor sirve a los requisitos no funcionales que el negocio realmente necesita.

INTERACCIÓN

¿Qué estilo predomina en su proyecto de CodeF@ctory?

En equipos, identifiquen:

1. ¿Qué estilo arquitectónico gobierna la mayoría de las decisiones estructurales de su proyecto? (flujo de datos, datos compartidos, llamada y retorno, independiente, datos virtuales)
2. ¿Qué patrón(es) del catálogo POSA ya vistos reconocen dentro de ese estilo?
3. ¿Hay alguna parte del sistema que use un estilo distinto al dominante? ¿Cuál y por qué?

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. Un **estilo arquitectónico** (Shaw y Garlan, Carnegie Mellon University, 1994–1996) es una categoría amplia que restringe el vocabulario de elementos de diseño y sus interacciones a nivel de sistema completo; un **patrón** es una solución concreta que vive dentro de un estilo.
2. Shaw y Garlan propusieron cinco familias: flujo de datos, datos compartidos, llamada y retorno, independientes y orientados a datos virtuales.
3. Los quince patrones POSA vistos en la Unidad 2 se agrupan mayoritariamente en llamada y retorno (nueve de quince), con flujo de datos, datos compartidos e independientes cubriendo el resto.
4. Ningún sistema real usa un solo estilo puro — la mayoría son híbridos; identificar el estilo dominante ayuda a razonar sobre el sistema sin negar esa mezcla.
5. Elegir un estilo arquitectónico es una decisión de negocio temprana y costosa de revertir, que debe basarse en los atributos de calidad prioritarios (ISO 25010), no en preferencia o moda.

Con esta sesión cerramos la Unidad 2 completa: catálogo de patrones + mapa de estilos que los organiza.

CIERRE

Para la próxima semana

- Repasar el mapa completo de estilos y confirmar en qué familia ubicarían cada patrón visto, sin mirar la tabla.
- Tener listo el proyecto de CodeF@ctory con su estilo arquitectónico dominante identificado y justificado.

La Semana 10 continúa la Unidad 2 con los Principios de Diseño de Software: SOLID, DRY, KISS y YAGNI.

